Web Reputational Analysis

Documentazione di Progetto

Progetto per la raccolta, normalizzazione ed esportazione dati

da fonti web eterogenee per analisi reputazionale

3 aprile 2026

Indice

1 Scopo del progetto **3**

2 Obiettivi **3**
2.1 Obiettivi funzionali . . . 3
2.2 Obiettivi non funzionali . . . 3

3 Ambito del progetto **4**
3.1 In scope . . . 4
3.2 Out of scope nella fase iniziale . . . 4

4 Stakeholder e utenti **4**

5 Fonti dati **5**
5.1 Google Search API . . . 5
5.2 YouTube Data API . . . 5
5.3 NewsAPI . . . 5
5.4 Wikipedia . . . 6
5.5 GDELT DOC 2.0 . . . 6

6 Requisiti funzionali **6**
6.1 RF-01 — Configurazione target . . . 6
6.2 RF-02 — Gestione query multiple . . . 6
6.3 RF-03 — Raccolta multi-fonte . . . 6
6.4 RF-04 — Persistenza raw . . . 6
6.5 RF-05 — Normalizzazione . . . 7
6.6 RF-06 — Pulizia . . . 7
6.7 RF-07 — Deduplicazione base . . . 7
6.8 RF-08 — Export . . . 7
6.9 RF-09 — Logging . . . 7
6.10 RF-10 — Fault tolerance parziale . . . 7

7 Requisiti non funzionali **7**
7.1 RNF-01 — Modularità . . . 7
7.2 RNF-02 — Estendibilità . . . 7
7.3 RNF-03 — Manutenibilità . . . 7
7.4 RNF-04 — Osservabilità . . . 7
7.5 RNF-05 — Portabilità . . . 7

8 Architettura logica **8**

9 Architettura del progetto **8**

10 Schema dati unificato **9**
10.1 Note sullo schema . . . 10

11 Convenzioni sui dati **10**
11.1 Date . . . 10
11.2 URL . . . 10
11.3 Domini . . . 10
11.4 Gestione dei valori null . . . 10
11.5 Liste nel CSV . . . 10

12 Strategia di deduplicazione **11**
12.1 Deduplicazione pre-ingestion in Python . . . 11
12.2 Deduplicazione avanzata downstream su IBM . . . 11

13 Logging e tracciabilità **11**

14 Gestione degli errori **12**

15 Sicurezza e configurazione **12**

16 Output attesi **12**
16.1 Output raw . . . 12
16.2 Output processed . . . 12
16.3 Output final . . . 12

17 Piano di sviluppo **12**
17.1 Fase 1 — Documentazione . . . 12
17.2 Fase 2 — Setup progetto . . . 13
17.3 Fase 3 — Implementazione collectors . . . 13
17.4 Fase 4 — Processors . . . 13
17.5 Fase 5 — Export . . . 13
17.6 Fase 6 — Test e validazione . . . 13
17.7 Fase 7 — Integrazione downstream . . . 14

18 Criteri di accettazione **14**

19 Rischi principali **14**

20 Decisioni progettuali iniziali **14**

21 Prossimi documenti da produrre **14**

22 Conclusione **15**

1 Scopo del progetto

L'obiettivo del progetto è realizzare una pipeline professionale di raccolta, normalizzazione ed esportazione dati per attività di **web reputational analysis** su personaggi pubblici, brand o altre entità di interesse.

Il sistema dovrà interrogare più fonti eterogenee, consolidare i risultati in uno **schema dati unificato** ed esportare dataset strutturati in formato **JSON** e **CSV**, in modo da renderli utilizzabili in una fase successiva all'interno di **IBM Cloud Pak for Data**.

Il progetto è pensato per essere:

- modulare;
- estendibile;
- manutenibile;
- compatibile con pipeline ETL successive;
- adatto a un contesto professionale.

2 Obiettivi

2.1 Obiettivi funzionali

Il sistema dovrà:

- raccogliere risultati da più fonti online;
- supportare query multiple per lo stesso target;
- uniformare i dati provenienti da API diverse;
- salvare dati grezzi, processati e finali;
- produrre output compatibili con pipeline ETL e Data Warehouse;
- consentire future estensioni con nuove fonti e nuove metriche.

2.2 Obiettivi non funzionali

Il sistema dovrà garantire:

- architettura modulare;
- codice manutenibile e testabile;
- logging chiaro e centralizzato;
- separazione netta tra raccolta, trasformazione ed esportazione;
- facilità di configurazione;

- robustezza rispetto a errori parziali delle singole fonti.

3 Ambito del progetto

3.1 In scope

Rientrano nell'ambito della prima fase del progetto:

- Google Search API;
- YouTube Data API;
- NewsAPI;
- Wikipedia tramite libreria Python;
- GDELT DOC 2.0;
- normalizzazione dati;
- deduplicazione base pre-ingestion;
- export JSON e CSV;
- predisposizione per utilizzo su IBM Cloud Pak for Data.

3.2 Out of scope nella fase iniziale

Non rientrano nella prima fase:

- dashboard frontend;
- sentiment analysis avanzata;
- NLP complesso;
- scraping non ufficiale di social network;
- entity resolution avanzata;
- deduplicazione semantica fuzzy avanzata.

4 Stakeholder e utenti

Gli attori principali del progetto sono:

- **Sviluppatore / Analista tecnico**: progetta, implementa e mantiene la pipeline.
- **Utente analista**: esegue raccolte dati su target specifici.
- **Sistema downstream**: IBM Cloud Pak for Data, che userà gli output del progetto per elaborazioni successive.

5 Fonti dati

5.1 Google Search API

Uso previsto: recupero di risultati web generali.

Dati attesi:

- titolo;
- snippet;
- URL;
- rank;
- eventuali metadati disponibili.

5.2 YouTube Data API

Uso previsto: recupero di video, canali e contenuti associati al target.

Dati attesi:

- titolo;
- descrizione;
- channel title;
- publishedAt;
- videoId o channelId;
- URL.

5.3 NewsAPI

Uso previsto: raccolta di articoli di news strutturati.

Dati attesi:

- source;
- author;
- title;
- description;
- URL;
- publishedAt;
- content, se disponibile.

5.4 Wikipedia

Uso previsto: recupero di contesto enciclopedico statico.

Dati attesi:

- titolo pagina;
- summary;
- URL;
- eventuali metadati di supporto.

5.5 GDELT DOC 2.0

Uso previsto: raccolta di articoli media e copertura giornalistica globale.

Dati attesi:

- titolo;
- URL;
- language;
- seendate;
- tone, se disponibile;
- sourcecountry, se disponibile.

6 Requisiti funzionali

6.1 RF-01 — Configurazione target

Il sistema deve permettere di definire un target principale, ad esempio una persona, un brand o un'organizzazione.

6.2 RF-02 — Gestione query multiple

Il sistema deve accettare una lista di query associate allo stesso target.

6.3 RF-03 — Raccolta multi-fonte

Il sistema deve poter interrogare selettivamente una o più fonti configurate.

6.4 RF-04 — Persistenza raw

Il sistema deve poter salvare i payload grezzi delle fonti per debugging, audit e verifiche successive.

6.5 RF-05 — Normalizzazione

Il sistema deve convertire tutti i risultati ottenuti in uno schema unico.

6.6 RF-06 — Pulizia

Il sistema deve normalizzare date, stringhe e valori nulli.

6.7 RF-07 — Deduplicazione base

Il sistema deve eliminare duplicati evidenti, almeno per URL e combinazioni titolo più dominio.

6.8 RF-08 — Export

Il sistema deve esportare i risultati finali in JSON e CSV.

6.9 RF-09 — Logging

Il sistema deve registrare esecuzione, errori e statistiche principali.

6.10 RF-10 — Fault tolerance parziale

Un errore su una singola fonte non deve bloccare l'intera pipeline.

7 Requisiti non funzionali

7.1 RNF-01 — Modularità

Ogni fonte deve essere implementata in un collector separato.

7.2 RNF-02 — Estendibilità

L'aggiunta di una nuova fonte deve richiedere modifiche minime.

7.3 RNF-03 — Manutenibilità

La logica di business deve essere separata dalla logica infrastrutturale.

7.4 RNF-04 — Osservabilità

Il sistema deve produrre log leggibili, tracciabili e utili al debugging.

7.5 RNF-05 — Portabilità

Il progetto deve poter essere eseguito in ambiente locale ed essere integrabile successivamente in pipeline più ampie.

8 Architettura logica

La pipeline logica del sistema è la seguente:

```
Input target/query
   |
   v
Collectors (API esterne)
   |
   v
Persistenza raw
   |
   v
Normalizer
   |
   v
Cleaner
   |
   v
Deduplicator
   |
   v
Validator
   |
   v
Exporters
   |
   v
Dataset JSON/CSV
   |
   v
IBM Cloud Pak for Data
```

Questa struttura separa in modo chiaro:

- la fase di **estrazione**;
- la fase di **trasformazione**;
- la fase di **caricamento** / **esportazione**.

9 Architettura del progetto

La struttura del repository prevista è la seguente:

```
web_reputational_analysis/
|
|-- main.py
|-- config.py
|-- requirements.txt
```

```
|-- README.md
|
|-- collectors/
|   |-- google_collector.py
|   |-- youtube_collector.py
|   |-- news_collector.py
|   |-- wikipedia_collector.py
|   `-- gdelt_collector.py
|
|-- processors/
|   |-- normalizer.py
|   |-- cleaner.py
|   |-- deduplicator.py
|   `-- validator.py
|
|-- exporters/
|   |-- json_exporter.py
|   `-- csv_exporter.py
|
|-- utils/
|   |-- helpers.py
|   |-- logger.py
|   `-- dates.py
|
|-- data/
|   |-- raw/
|   |-- processed/
|   `-- final/
|
`-- tests/
    |-- test_collectors.py
    |-- test_normalizer.py
    `-- test_deduplicator.py
```

10 Schema dati unificato

Ogni record normalizzato dovrà aderire al seguente schema logico:

```
{
  "source_type": "",
  "source_name": "",
  "target_entity": "",
  "query": "",
  "title": "",
  "snippet": "",
  "content": "",
  "url": "",
  "domain": "",
```

```
  "author": "",
  "published_at": "",
  "retrieved_at": "",
  "language": "",
  "country": "",
  "rank": null,
  "views_count": null,
  "likes_count": null,
  "comments_count": null,
  "engagement_score": null,
  "keywords_found": [],
  "sentiment_stub": null,
  "raw_payload": {}
}
```

10.1 Note sullo schema

- Non tutti i campi saranno valorizzati da tutte le fonti.
- I campi mancanti dovranno essere uniformati secondo una convenzione unica.
- Il campo `raw_payload` è utile nel JSON, ma può essere escluso o serializzato nel CSV.

11 Convenzioni sui dati

11.1 Date

Tutte le date devono essere convertite in formato ISO 8601 quando possibile.

11.2 URL

Gli URL devono essere salvati in forma completa e validata.

11.3 Domini

Il dominio deve essere estratto dall'URL e salvato separatamente.

11.4 Gestione dei valori null

Valori non disponibili devono essere uniformati secondo una convenzione unica definita a livello di progetto.

11.5 Liste nel CSV

Campi lista, come `keywords_found`, devono essere serializzati in una stringa delimitata, ad esempio con il carattere `;`.

12 Strategia di deduplicazione

12.1 Deduplicazione pre-ingestion in Python

Questa fase ha l'obiettivo di eliminare i duplicati evidenti prima dell'export.

Criteri iniziali previsti:

- URL identico;
- titolo e dominio identici;
- identificatore specifico della fonte, se disponibile.

12.2 Deduplicazione avanzata downstream su IBM

Eventualmente, in una fase successiva, si potrà introdurre:

- similarità testuale;
- entity resolution;
- clustering semantico;
- consolidamento cross-source più sofisticato.

La scelta corretta è quindi a due livelli:

1. deduplicazione base nel codice Python;
2. deduplicazione avanzata in ambiente IBM, se necessaria.

13 Logging e tracciabilità

Ogni esecuzione dovrebbe registrare almeno:

- timestamp di avvio e di fine;
- target analizzato;
- query eseguite;
- fonte interrogata;
- numero di record raccolti;
- numero di record scartati;
- errori riscontrati.

Il logging dovrà essere sufficientemente chiaro da supportare:

- debugging;
- audit;

- monitoraggio del comportamento della pipeline.

14 Gestione degli errori

La strategia di error handling prevista è la seguente:

- `try/except` per ogni collector;
- fallback controllato quando opportuno;
- continuazione della pipeline in caso di errore parziale;
- log degli errori con contesto minimo sufficiente.

Questo approccio evita che il fallimento di una singola fonte comprometta l'intera esecuzione.

15 Sicurezza e configurazione

Le seguenti decisioni devono essere considerate vincolanti:

- le API key non devono essere hardcodate nel codice sorgente;
- le credenziali devono essere gestite tramite variabili ambiente o file `.env` non versionato;
- i file sensibili devono essere esclusi dal repository tramite `.gitignore`.

16 Output attesi

16.1 Output raw

Payload grezzi per singola fonte e query, salvati per fini di debug e audit.

16.2 Output processed

Record già normalizzati e puliti, utili per verifiche intermedie.

16.3 Output final

Dataset finale deduplicato esportato in:

- `output.json` oppure `output.jsonl`;
- `output.csv`.

17 Piano di sviluppo

17.1 Fase 1 — Documentazione

- definizione obiettivi;
- definizione architettura;

- definizione schema dati;
- definizione requisiti.

17.2 Fase 2 — Setup progetto

- struttura cartelle;
- requirements;
- configurazione;
- logger.

17.3 Fase 3 — Implementazione collectors

- Google;
- YouTube;
- NewsAPI;
- Wikipedia;
- GDELT.

17.4 Fase 4 — Processors

- normalizer;
- cleaner;
- deduplicator;
- validator.

17.5 Fase 5 — Export

- JSON;
- CSV.

17.6 Fase 6 — Test e validazione

- test unitari base;
- test di integrazione minimi;
- verifica degli output.

17.7 Fase 7 — Integrazione downstream

- caricamento su IBM Cloud Pak for Data;
- controlli ETL successivi.

18 Criteri di accettazione

Il progetto può considerarsi correttamente avviato quando:

- la documentazione è approvata;
- la struttura del progetto è definita;
- lo schema dati unificato è stabile;
- almeno una pipeline end-to-end funziona da input target a export JSON/CSV.

19 Rischi principali

I rischi principali del progetto sono:

- limiti e quote delle API;
- differenze di schema tra fonti;
- dati incompleti o incoerenti;
- duplicati cross-source;
- qualità variabile delle query;
- instabilità dei metadati restituiti da fonti diverse.

20 Decisioni progettuali iniziali

Le decisioni iniziali già fissate sono:

1. architettura modulare a componenti separati;
2. schema dati unico come centro del progetto;
3. deduplicazione base lato Python, avanzata eventuale lato IBM;
4. persistenza dei dati raw per debug e audit;
5. output finale compatibile con pipeline ETL successive.

21 Prossimi documenti da produrre

Dopo questo documento, i documenti da produrre sono:

1. specifica tecnica dettagliata dei moduli;

2. definizione del file di configurazione;
3. contratto dati dei collector;
4. schema del record normalizzato con tipi e vincoli;
5. piano di test;
6. README operativo.

22 Conclusione

Questo documento definisce la base architetturale e organizzativa del progetto di Web Reputational Analysis.

La priorità corretta, prima di scrivere codice, è:

1. fissare documentazione e requisiti;
2. bloccare lo schema dati;
3. definire responsabilità dei moduli;
4. solo dopo iniziare l'implementazione.

Questa scelta riduce il rischio di produrre codice disordinato, poco estendibile o difficile da integrare in pipeline successive.